课程笔记：深度研究（Deep Research Agents）

Bowen-mantou（整理）

2026-10-23

视频作者/频道：B 站「常青藤名校课」· CMU 11-768 AI Agents（英文原声字幕版）
发布日期：2026-09-09
视频时长：72:25
视频链接：https://www.bilibili.com/video/BV1a6Yx62EH4?p=24
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 引言：什么是深度研究 agent **2**
1.1 从「一次检索」到「反复搜索与综合」 2
1.2 Deep research agent 的一次迭代 2
1.3 本章小结 2

2 评测：从经典 QA 到 deep research benchmark **2**
2.1 经典 QA benchmark 的四个局限 2
2.2 BrowseComp：多条件、多跳、短答案 3
2.3 BrowseComp-Plus：固定语料与可复现评测 3
2.4 开放式长答案与 rubric 评测 3
2.5 本章小结 4

3 建模：如何训练 deep research agent **4**
3.1 推理链路回顾 4
3.2 三阶段训练配方 4
3.3 用知识图谱构造多跳问题 4
3.4 短答案任务：RLVR 与 GRPO 5
3.5 工程：让长轨迹 RL 跑得动 5
3.6 开放式任务：rubric-based RL 与 DR Tulu 5
3.7 本章小结 6

4 检索：为 agent 构建搜索基础 **6**
4.1 本地检索与黑盒 API 6
4.2 Embedding 检索的复习与训练目标 6
4.3 Agent 检索的特殊性：检索器只看得到最后一个 query 6
4.4 本章小结 7

5 总结与延伸 **7**

1 引言：什么是深度研究 agent

这一讲由 CMU LTI 的新晋助理教授 Akari Asai 主讲（她称这是自己在 CMU 的第一堂课），主题是 **deep research agents**——面向复杂信息需求的搜索型 agent。她按「评测（evaluation）→ 建模（modeling）→ 检索（retrieval）」三块组织全讲，并在最后讨论开放问题。

1.1 从「一次检索」到「反复搜索与综合」

课堂先对比了两个问题。第一个是「我在 CMU 的办公室是几号?」——只要搜一次、找到系里个人主页上的信息就能回答；用一个简单的「检索增强生成（RAG）」pipeline，让模型读一篇文档并抽取答案即可。整个过程是：**一次搜索**、**一篇正确网页**、**直接作答**。

第二个问题是「AI agent 能像人类专家一样综合科学文献吗?」——没有哪一篇文档能直接回答它，必须反复查看多篇论文与多个网站、比较不同研究的结论。此时需要的是 **deep research agent**：它**反复规划**、**调用外部搜索**、**反思结果**，再决定是否继续搜索，最后综合出带引用的长答案。

1.2 Deep research agent 的一次迭代

把上面的过程拆开，agent 在每一轮做四件事：**计划（plan）**——我需要先找到哪些信息，例如文献综合的研究、语言模型与人类专家对比的研究；**搜索（search）**——生成搜索 query，交给外部搜索系统，得到一组文档；**反思（reflect）**——这些结果是否回答了问题? 还缺什么? **修订（revise）**——改写 query 再搜。如此迭代若干轮，收集到足够信息后**综合出最终答案并附引用**。

这条链路也带来了三类新的研究问题：如何**评测**这类 agent；如何**训练**它（从示范到 RL）；以及如何为它构建更好的**检索系统**——这正是本讲的三大部分。

1.3 本章小结

Deep research agent 面向「没有单一文档能回答」的复杂信息需求：它反复「规划—搜索—反思—修订」，最后综合多来源并给出引用。评测、训练与检索是支撑它的三根柱子。

2 评测：从经典 QA 到 deep research benchmark

2.1 经典 QA benchmark 的四个局限

课堂以一个经典问答 benchmark（Natural Questions，来自真实 Google 搜索 query、用当年维基百科标注答案）为例，指出它与今天 deep research 用法的差距，归纳成四个维度：

1. **任务复杂度**：大多是单跳查找（simple lookup），一个事实能在一次搜索里找到；
2. **领域**：多为通用领域（体育、娱乐），而 deep research 越来越多用于专家领域（金融、医疗、科研）；
3. **答案质量**：多为短实体答案（short-form），而 deep research 常需要长文开放式综合；
4. **证据支持**：旧 benchmark 不检查答案是否有可靠证据支撑，而信息类问答恰恰需要知道「答案从哪来」。

2.2 BrowseComp：多条件、多跳、短答案

OpenAI 的 **BrowseComp** 是当前最常用的 deep research benchmark 之一：包含 1000 多道人工撰写的题目。它的构造方式很巧妙——作者**从已知答案倒推**：先选定一个人物、事件或物品作为目标答案，再收集一组线索（clue），把多条线索组合成一道复杂问题。这样问题的最终答案仍是短实体（便于自动判定是否正确），但解题必须经过多次搜索与多条件筛选。

题目还经过多道验证：先用语言模型加搜索确认「不能只靠模型本身或 5 次 Google 搜索答对」，再让人类确认「10 分钟内答不对」。课堂展示的一道题要求同时满足五条线索（虚构角色、1970 年代电视剧、少于 50 集等），只有同时满足所有条件的才是正确答案。

结果很能说明难度：在带浏览工具的设置下，GPT-4o 只有 0.6%、GPT-4o+browsing 1.9%、GPT-4.5 0.9%、o1（medium）9.9%；而专门为 deep research 训练的 OpenAI Deep Research 达到 51.5%，人类参考正确率是 25.3%。课堂引用原论文的结论：即使有外部搜索，当时的强模型仍大幅落后于专用系统，而人类专家在该 benchmark 上也只有约 25%。

课堂还引用了 Tongyi DeepResearch 的消融，说明这类任务的**长度尺度**：把最大上下文长度从 8K 提到 128K，准确率从接近 0 提升到约 40% 量级；而工具调用上限设到 128 次时效果最好。也就是说，高难度 deep research 需要**上百次搜索调用与很长的轨迹**。这带来一个很实际的问题：用真实 web 搜索 API 既慢又贵，而且 Google 的结果几乎每天变化，评测难以复现。

2.3 BrowseComp-Plus：固定语料与可复现评测

BrowseComp-Plus 解决复现性问题：它**固定 agent 可检索的文档集合**，把「开放 web 搜索」换成「在受控语料里检索」。构造流程是：从问题与答案出发，用语言模型找出可能有用文档、再由人工核验；然后混入「看起来有用但其实无关」的负样本文档，最终形成一个约 10 万篇文档、约 830 道题的语料库。这样评测可重复，也能单独研究检索系统（这正是第 4 节的内容）。

除了通用域，课堂还介绍了若干专家领域 benchmark：**FinSearchComp**（金融；由 MBA/金融从业者出题，使用金融数据库并用计算工具核对答案，据课堂口径有 635 道深度研究题、经过多轮专家审核）、**MedBrowseComp**（医疗；查检验、药物、监管事实）、**AutoResearchBench**（科研；找出满足多条条件的那篇论文）。共同结论是：高质量的专家域 deep research benchmark 必须靠**专家出题与核验**。

2.4 开放式长答案与 rubric 评测

短答案可以用 exact match 判定，但很多 deep research 任务要求**长文开放式综合**。传统做法是标注参考并用相似度指标（ROUGE、BERTScore、BLEURT 等）比较模型输出；课堂指出这类指标很不可靠：一项针对长文问答的实证研究让专家在两条回答中选更好的，再与指标比较，结果这些指标与专家偏好的**一致率只有 58%（ROUGE）、57%（BERTScore）、62%（BLEURT）**，只比随机 50% 略高。

替代方案是 **rubric（评分量表）评测**：让专家写一组评价标准，每条标准有具体描述与权重；对每条标准，用一个语言模型判断模型答案是否满足，再对满足项加权求和。课堂（基于其所在团队的 ScholarQA 工作）报告：只要 rubric 质量高，专家之间的一致性约 **80%**，语言模型与专家之间约 **79%**——这意味着 rubric 可以在保证质量的前提下被语言模型大规模使用。

代价是 rubric 很贵：课堂说一个博士专家标注一条 rubric 大约要 **1 小时**，难以规模化。于是有工作尝试自动生成 rubric：例如 ResearchQA 从综述论文出发让语言模型生成问题与 rubric，可扩到约 2.1 万道题、16 万条 rubric；但人工审计发现约 **15%** 的自动 rubric 存在错误——引用不存在

的论文、重复问题、或标准过于空泛以至于无法区分好坏答案。结论是：自动 rubric 必须**用检索/来源做 grounding**，并经过审计。

此外，**证据质量**本身也值得评测：DeepResearch Bench 提出 FACT 框架，抽取答案中的「claim + 引用 URL」，逐条检查该 URL 是否真的支持这句话；课堂用一道题的例子说明：两条 claim 引用同一个网页，只有一条被支持，则该题的引用准确率为 50%。不过课堂也指出：FACT 检查「引用是否支持 claim」，但不检查「被引用的来源本身是否可靠」，后者仍是开放问题。

2.5 本章小结

评测经历了从「单跳、短答案、无证据」到「多跳、专家域、长答案、要证据」的升级：BrowseComp 用多条件问题与逆向构造把难度拉满，BrowseComp-Plus 用固定语料换来可复现；开放式答案要靠 rubric（专家—语言模型一致性约 79%）而不是相似度指标；引用质量可用 FACT 框架逐条核验。课堂的选用建议是：短答案验证型任务用 BrowseComp（质量高但贵）；要降本、要研究检索用 BrowseComp-Plus；开放式任务用 DeepResearch Bench；专家域可从 FinSearchComp 入手。

3 建模：如何训练 deep research agent

3.1 推理链路回顾

给定一个复杂问题，agent 先产生思考 token 与计划，再生成工具调用（搜索 query），外部搜索系统返回文档并进入上下文；模型基于新证据继续推理，判断是否继续搜索；经过若干轮后产出最终答案与引用。这一节讨论**怎么训练**这样的模型。

3.2 三阶段训练配方

课堂以阿里巴巴通义团队的 DeepResearch 为例，给出常见三阶段配方：

1. **Mid-training**：让 base model 在大规模数据上接触 agent 行为，成为「agent-ready」的底座；
2. **SFT**：让学生模型模仿强教师生成的（成功）轨迹；
3. **RL**：让策略模型从**自己的尝试**中学习，用可验证奖励或 rubric 奖励更新。

三个阶段都依赖高质量数据：mid-training 与 SFT 需要高质量**轨迹**，RL 需要有可验证答案的**问题**。数据从哪里来？

3.3 用知识图谱构造多跳问题

课堂用知识图谱（knowledge graph）解释训练数据的规模化构造：实体是节点、关系是有向边。例如「Arun Newell」与「Carnegie Mellon」两个节点之间有一条「他是该校教授」的边；从这个实体出发，还可以连到「1975 年图灵奖」「与 Herbert Simon 合著 Human Problem Solving」，而 Simon 又连到「诺贝尔经济学奖」。

做法是：在知识图谱上做**随机游走**，采样一组相连的事实，把它们组合成一道多跳复杂问题，并以「被查询的实体」作为目标答案。这与 BrowseComp 的构造方式同源——某种意义上是在**为 benchmark 反向工程训练数据**。课堂也诚实指出：这种构造方式训练出的模型能否迁移到其他类型的 deep research benchmark，仍是开放问题；但它提供了一条可扩展的高质量数据构造路径。

有了问题与答案，还需要教师轨迹：让更强的模型（专有模型或更大的开源模型）为每题生成多条轨迹，按「最终答案是否匹配 / 格式是否合法」筛掉失败轨迹，只保留成功轨迹做 SFT。有的系统更早一步做 **agentic mid-training**：把大量 agent 轨迹混入持续预训练数据，再做同样的下游 SFT；课堂给的对比显示，同样的 SFT 数据下，mid-trained 底座的最终表现明显更好（尤其在复杂 benchmark 上）。

3.4 短答案任务：RLVR 与 GRPO

对短答案、可验证的任务，可以直接用 **RLVR**：策略模型对同一题生成多条轨迹，用 exact match 或语言模型 judge 判断最终答案是否正确，得到奖励，再用策略梯度更新。课堂明确提到这条路线最早在搜索 agent 上的尝试之一就是 Search-R1 一类工作。

以 GRPO 为例：同一道题生成一组轨迹（如 3 条），其中两条答对、一条答错，组均值以上的轨迹被强化、以下的被抑制。课堂用「Herbert Simon（奖励 0）vs Allen Newell（奖励 1，出现两次）」的例子说明组内相对优势：答对的轨迹优势为正、答错的为负，更新时把概率质量推向正确尝试。实际训练还会加 clipping（限制单步更新幅度）与 KL 正则（限制偏离初始策略太远），这两项在大规模训练时对稳定性很重要。

3.5 工程：让长轨迹 RL 跑得动

Agentic RL 很贵也不稳定：轨迹长达上百次搜索调用，每次调用要等外部返回。课堂介绍了几条工程手段：**异步 rollout**（一条轨迹在等工具时，其他轨迹继续生成，完成后统一打分组 batch）、**缓存搜索结果**、更健壮的 API 客户端与重试、以及用**本地索引/模拟环境**替代真实 API。训练曲线通常表现为奖励上升而策略熵保持相对稳定——这是训练健康的一个观察点。

3.6 开放式任务：rubric-based RL 与 DR Tulu

短答案有 exact match，但开放式研究报告没有二值正确性：一份好的文献综述需要覆盖度、相关性、事实性与恰当引用。课堂介绍了其团队的 **DR Tulu**（Reinforcement Learning with Evolving Rubrics for Deep Research），核心是**让 rubric 与策略一起演化**。

流程：先用外部搜索 **grounding** 出一组初始 rubric；训练时对每题采样 8 条轨迹；用语言模型**对比多条轨迹**，生成新的「正 rubric」（奖励好的行为）与「负 rubric」（惩罚具体错误）；维护一个 rubric 缓冲区，用它给最好的回答打分并更新策略。关键在于：反馈不是训练开始时固定的，而是**随策略探索不断捕捉新的区分**——策略变强后，rubric 也能识别更细的质量差异。

课堂报告的消融很值得记住：用**随机奖励**做 online RL 看不到任何提升；用**初始 search-grounded rubric** 明显更好；而 evolving rubric 在训练更久时收益更大——说明「初始 rubric 质量」与「rubric 的适应性」两者都重要。另一个关键发现是 **SFT 是 RL 的热身**：从 base 模型直接 RL（600 步）只有约 40%，而只用约 5% 的 SFT 数据做「欠训练 SFT」后，同样步数可再提升约 10 个百分点；完整 SFT 则给出更强的 RL 起点。

最终，DR Tulu-8B 在四个 deep research benchmark 的平均分上，以远低于专有系统的推理成本取得有竞争力的表现（课堂的成本—性能图：横轴是每查询成本，纵轴是平均分；DR Tulu-8B 位于左上角）。

rubric 生成不能裸用语言模型

课堂引用 ResearchQA 的审计：纯语言模型生成的 rubric 约 15% 有错（引用不存在的论文、重复问题、标准空泛）。可用的 rubric 需要外部证据 grounding + 审计；rubric 生成模型越强越好（课堂口径：GPT-4.1 与 Qwen3 系做 rubric 生成约差 10 个点），开源 rubric 模型仍是开放问题。

3.7 本章小结

训练配方是 mid-training → SFT → RL：用知识图谱随机游走构造多跳问答、用强教师轨迹做 SFT、用 RLVR/GRPO 从自身尝试学习。开放式报告用 rubric-based RL，DR Tulu 让 rubric 与策略共同演化；SFT 是 RL 的热身，少量 SFT 也能显著抬高 RL 起点；rubric 必须 grounded 且审计。

4 检索：为 agent 构建搜索基础

4.1 本地检索与黑盒 API

课堂把 deep research agent 可用的检索分成两类：

- **本地检索（local search）**：已知语料时，可在本地建词法检索（BM25）或神经 embedding 检索（训练过的相似度模型）。例如 Search-R1 一类工作用 2018 年维基百科语料 + embedding 检索器。
- **黑盒 API**：开放 web 场景用 web search API（如 Serper）或浏览 API；科研场景用 Semantic Scholar、Google Scholar 等论文搜索 API。

选择取决于任务：要做好 BrowseComp/公开 web 题，需要真实 web 搜索；要做 BrowseComp-Plus 这类受控语料评测，本地索引更可复现也更便宜。

4.2 Embedding 检索的复习与训练目标

标准 embedding 检索分两步：离线用文档编码器把所有文档编码成向量、建向量索引；在线用查询编码器编码 query，做近邻检索。训练常用**对比学习**：每个 query 配一个正文档与若干负文档，目标函数提高正文档相对负文档的得分：

$$\mathcal{L} = -\log \frac{\exp\big(s(q, d^+)\big)}{\exp\big(s(q, d^+)\big) + \sum_{d^-} \exp\big(s(q, d^-)\big)},$$

其中 q 是查询，d^+ 是正文档，d^- 是负文档，$s(\cdot,\cdot)$ 是相似度打分（如内积或余弦）。负样本可以来自 **in-batch negatives**（同一 batch 里其他 query 的正文档充当负例，DPR 的经典做法），也可以由较弱的检索器（如 BM25）挖出**困难负例**再混合。

4.3 Agent 检索的特殊性：检索器只看得到最后一个 query

标准检索器是为「人类用户的一次查询」设计的；但 deep research agent 在每次检索时拥有**很丰富的历史**：原始任务、之前的推理 token、之前的 query、之前搜索的结果。课堂指出一个关键问题：现有系统通常把**当前 query** 单独交给检索器，历史信息被丢掉了。

课堂介绍了其团队的 **AgentIR**：把「当前推理 token + query」一起输入检索器，并针对这种「推理感知（reasoning-aware）检索」训练 embedding。消融显示两点都重要——只在推理时拼接推理 token 有小幅提升，而对检索器做针对性训练能带来更大提升；两者结合效果最好。另一个有意思的发现是**上下文不是越多越好**：把整条轨迹都塞给检索器反而变差（上下文太长），而只用「上一步的推理 token」最有效——它恰好总结了任务与上一步搜索结果。只用历史 query（不含搜索结果）帮助有限。

课堂报告：AgentIR 在 BrowseComp-Plus 上同时提升准确率、降低平均搜索调用次数；它只有 4B，比 Qwen3-8B 这类通用 embedding 模型更小；训练数据只来自 BrowseComp-Plus 的少量轨迹，却能在不同 agent 模型与不同 benchmark 上泛化——说明「为 agent 的检索需求专门训练检索器」是一条值得投入的方向。

4.4 本章小结

检索要么在本地受控语料上建 BM25/embedding 索引（可复现、便宜），要么调黑盒 web/论文搜索 API（开放但慢且不可复现）。embedding 用对比学习训练，负样本可用 in-batch 与 BM25 困难负例。Agent 检索的特殊性在于历史丰富而检索器只看最后一个 query；AgentIR 把「当前推理 + query」交给检索器并专门训练，在更少搜索次数下取得更高准确率，且小模型也能泛化。

5 总结与延伸

主讲人最后用四点总结全讲：(1) **任务**——deep research 是「迭代搜索 + 证据综合」，不是单次检索问答；(2) **评测**——要同时评搜索复杂度、答案质量、引用支撑，以及 rubric 本身的质量；(3) **训练**——用可验证问题构造任务与轨迹，从示范学习（mid-training/SFT），再从自身尝试中学习（RL）；(4) **检索**——要为 agent 当前的检索需求构建并训练检索器，而不是沿用为人类查询设计的检索器。

全讲可以压缩为六条：

1. Deep research agent 的循环是「规划 → 搜索 → 反思 → 修订 → 综合带引用答案」，适用于没有单一文档能回答的复杂信息需求。
2. 经典 QA benchmark 在四个维度上不够用（复杂度、领域、答案形态、证据支持）；BrowseComp 用逆构造的多条件问题把难度拉满，BrowseComp-Plus 用固定语料解决可复现性，FinSearch-Comp 等专家域 benchmark 依赖专家出题核验。
3. 开放式答案不能用相似度指标评（与专家偏好一致率仅 57%–62%），rubric 评测在高质量前提下专家—模型一致性约 79%；自动 rubric 有约 15% 错误，需要 grounding 与审计；引用质量可用 FACT 逐条核验。
4. 训练配方是 mid-training → SFT → RL：知识图谱随机游走构造多跳问题、强教师轨迹筛选后 SFT、RLVR/GRPO 从自身尝试学习；SFT 是 RL 的热身（少量 SFT 也能显著抬高起点）。
5. 开放式报告用 rubric-based RL，DR Tulu 让 rubric 与策略共同演化，用 8B 模型在低成本下取得有竞争力的平均分。
6. 检索是 deep research 的底座：本地 BM25/embedding 与黑盒 API 各有取舍；AgentIR 表明把「当前推理 + query」交给检索器并专门训练，可以在更少搜索次数下提高准确率。

一句话记住本讲

Deep research = 评测（多跳/专家域/长答案/证据）、训练（mid-training → SFT → 可验证或 rubric 的 RL）、检索（为 agent 的推理历史专门训练）三件事的组合；三者缺一，agent 都会在「搜不到、评不准、学不会」上卡住。

延伸思考：(1) 从系统角度看，deep research 的成本主要由「搜索调用次数」与「上下文长度」决定，而两者又由训练配方决定——这与第 9 讲「RL 的组大小、全对全错比例」一样，属于「采样预算设计」问题；(2) rubric 与第 8 讲 SFT 的「教师信号」、第 9 讲的「奖励设计」是同一条主线：开放式任务缺少可验证信号时，只能用「可审计的近似信号」，而近似信号的偏差会被策略利用（reward hacking 的另一种形态）；(3) retrieval 与 agent 的联合训练很可能是下一个热点——检索器需要理解 agent 的意图与历史，而 agent 又需要理解检索器的偏好，两者目前还是分开训练的。课堂在问答中还给了实操建议：从 8B 级模型（Qwen3-8B 一类）与稳定 web search API 起步，训练时用 BrowseComp-Plus 的本地索引（BM25 或 Qwen3-4B/8B embedding）压成本；embedding 选型可参考 MTEB；报告结果时最好在多个 agent 模型（如 GPT-OSS、GLM）上验证，而不是只调优单一 agent。